مشروع مادة خوارزميات التحسين

الشيفرة المصدرية الكاملة

جدولة المهام على آلات غير متجانسة

GA | PSO | ACO | Greedy

ملفات التجارب والاختبار	ملفات Python	لغة البرمجة
تجربة **54**	ملفات **3**	**Python 3.10+**

حول هذا الملف

يحتوي هذا الملحق على النص الكامل لملفات المشروع البرمجية، مع ترقيم للأسطر لتسهيل الدراسة والمراجعة. أُدرج الكود كما هو في ملفات Python الأصلية. لتشغيل المشروع، استخدم ملفات .py الموجودة في الحزمة المضغوطة، وليس مستند Word أو PDF.

طريقة تشغيل المشروع

```
python -m pip install -r requirements.txt
python -m unittest discover -s tests -v
python run_experiments.py
requirements.txt:  matplotlib>=3.5
```

الملفات المدرجة

1 **scheduler.py** — تعريف المسألة وGreedy وتنفيذ GA وPSO وACO

2 **run_experiments.py** — تنفيذ التجارب وتصدير النتائج والمنحنيات

3 **tests/test_scheduler.py** — الاختبارات الآلية للتأكد من صحة الكود

4 **requirements.txt** — المكتبة البرمجية المطلوبة

ملاحظة: التقرير الأكاديمي الأصلي مستقل في ملفي Word وPDF المرفقين بالمشروع.

1. الخوارزميات ودالة الهدف

scheduler.py

```python
"""Independent-task scheduling on unrelated parallel machines.

Each assignment[j] is a machine ID in range(num_machines).
Objective: minimize makespan, max over machines of summed processing times.
The metaheuristics are implemented directly, not imported from solver libraries.
All stochastic algorithms take an explicit seed for reproducibility.
"""

from __future__ import annotations

from dataclasses import dataclass
import math
import random
from typing import Sequence


@dataclass(frozen=True)
class Instance:
    """durations[j][m]: minutes to process task j on machine m."""

    name: str
    durations: tuple[tuple[float, ...], ...]

    def __post_init__(self) -> None:
        if not self.durations or not self.durations[0]:
            raise ValueError("Instance must include at least one task and machine")
        machine_count = len(self.durations[0])
        if any(len(row) != machine_count for row in self.durations):
            raise ValueError("All tasks must have times for the same machines")
        if any(not math.isfinite(t) or t <= 0 for row in self.durations for t in row):
            raise ValueError("Processing times must be positive finite numbers")

    @property
    def num_jobs(self) -> int:
        return len(self.durations)

    @property
    def num_machines(self) -> int:
        return len(self.durations[0])


def generate_instance(name: str, jobs: int, machines: int, seed: int) -> Instance:
    """Deterministic synthetic workshop with different speeds/affinities.

    Duration_ij = round(work_j / speed_i * affinity_ji + setup_ji, 2).
    Different task-machine pairs have distinct processing times.
    """
    if jobs < 1 or machines < 1:
        raise ValueError("jobs and machines must both be positive")
    rng = random.Random(seed)
    speeds = [rng.uniform(0.72, 1.38) for _ in range(machines)]
    work = [rng.randint(8, 30) for _ in range(jobs)]
    matrix = []
    for w in work:
        matrix.append(tuple(
            round(w / speed * rng.uniform(0.78, 1.28) + rng.uniform(0, 3), 2)
            for speed in speeds
        ))
    return Instance(name, tuple(matrix))


def machine_loads(instance: Instance, assignment: Sequence[int]) -> list[float]:
    if len(assignment) != instance.num_jobs:
        raise ValueError("Exactly one assignment is required per task")
    loads = [0.0] * instance.num_machines
    for job, machine in enumerate(assignment):
        if not isinstance(machine, int) or not (0 <= machine < instance.num_machines):
            raise ValueError(f"Invalid machine {machine} for task {job}")
        loads[machine] += instance.durations[job][machine]
    return loads


def fitness(instance: Instance, assignment: Sequence[int]) -> float:
    return max(machine_loads(instance, assignment))


def theoretical_lower_bound(instance: Instance) -> float:
    """Valid, sometimes loose bound; NOT a known optimum."""
    minima = [min(row) for row in instance.durations]
```

```
    return max(sum(minima) / instance.num_machines, max(minima))


def greedy_assignment(instance: Instance) -> list[int]:
    """LPT-like baseline: hardest jobs first, choose earliest finish."""
    n, m = instance.num_jobs, instance.num_machines
    order = sorted(range(n), key=lambda j: (-min(instance.durations[j]), j))
    loads = [0.0] * m
    assignment = [0] * n
    for j in order:
        machine = min(
            range(m), key=lambda k: (loads[k] + instance.durations[j][k], instance.durations[j][k], k)
        )
        assignment[j] = machine
        loads[machine] += instance.durations[j][machine]
    return assignment


@dataclass
class SearchResult:
    algorithm: str
    assignment: list[int]
    best_fitness: float
    history: list[float]  # initial population + one best-so-far value per iteration
    evaluations: int

    @property
    def convergence_iteration(self) -> int:
        """First recorded iteration reaching the final best fitness."""
        return next(i for i, v in enumerate(self.history) if abs(v - self.best_fitness) <= 1e-8)


def _mutate(chrom: list[int], machine_count: int, rng: random.Random,
            gene_rate: float, swap_rate: float) -> list[int]:
    mutated = chrom.copy()
    for j in range(len(mutated)):
        if rng.random() < gene_rate:
            old = mutated[j]
            # Always select a different machine, where possible.
            mutated[j] = (old + rng.randrange(1, machine_count)) % machine_count if machine_count > 1 else old
    if len(mutated) > 1 and rng.random() < swap_rate:
        a, b = rng.sample(range(len(mutated)), 2)
        mutated[a], mutated[b] = mutated[b], mutated[a]
    return mutated


def genetic_algorithm(instance: Instance, seed: int, iterations: int = 80,
                      population_size: int = 32, mutation_rate: float | None = None,
                      crossover_rate: float = 0.9, elite_count: int = 2,
                      tournament_size: int = 3) -> SearchResult:
    """Integer GA: tournament selection, uniform crossover, mutation, elitism."""
    if population_size < 3 or not (0 <= elite_count < population_size):
        raise ValueError("Invalid population/elitism settings")
    if iterations < 0:
        raise ValueError("Iterations cannot be negative")
    rng = random.Random(seed)
    n, m = instance.num_jobs, instance.num_machines
    mutation_rate = 1 / n if mutation_rate is None else mutation_rate
    if not 0 <= mutation_rate <= 1:
        raise ValueError("mutation_rate must be between 0 and 1")
    baseline = greedy_assignment(instance)
    pop = [baseline]
    # Start near a feasible greedy solution, with diverse global candidates.
    while len(pop) < population_size:
        if len(pop) % 4 != 0:
            candidate = baseline.copy()
            changes = rng.randint(1, max(2, n // 4))
            for j in rng.sample(range(n), min(changes, n)):
                candidate[j] = rng.randrange(m)
        else:
            candidate = [rng.randrange(m) for _ in range(n)]
        pop.append(candidate)
    scored = sorted(((fitness(instance, a), a) for a in pop), key=lambda x: x[0])
    evaluations = population_size
    history = [scored[0][0]]

    def tournament(items: list[tuple[float, list[int]]]) -> list[int]:
        return min((rng.choice(items) for _ in range(tournament_size)), key=lambda t: t[0])[1]

    for _ in range(iterations):
        new_pop = [a.copy() for _, a in scored[:elite_count]]
        while len(new_pop) < population_size:
            parent_a, parent_b = tournament(scored), tournament(scored)
            if rng.random() < crossover_rate:
                child = [parent_a[j] if rng.random() < 0.5 else parent_b[j] for j in range(n)]
            else:
```

```
                child = parent_a.copy()
            child = _mutate(child, m, rng, mutation_rate, 0.25)
            new_pop.append(child)
        scored = sorted(((fitness(instance, a), a) for a in new_pop), key=lambda x: x[0])
        evaluations += population_size
        history.append(scored[0][0])
    best_value, best_assignment = scored[0]
    return SearchResult("GA", best_assignment.copy(), best_value, history, evaluations)


def particle_swarm_optimization(instance: Instance, seed: int, iterations: int = 80,
                                swarm_size: int = 32, inertia: float = 0.68,
                                cognitive: float = 1.6, social: float = 1.6,
                                velocity_limit: float = 2.0,
                                exploration_rate: float = 0.012) -> SearchResult:
    """Discrete PSO via continuous positions rounded to machine IDs.

    A small random reset of coordinates preserves exploration in discrete space.
    """
    if swarm_size < 2 or iterations < 0:
        raise ValueError("Invalid PSO settings")
    rng = random.Random(seed)
    n, m = instance.num_jobs, instance.num_machines
    seed_solution = greedy_assignment(instance)

    def decode(pos: list[float]) -> list[int]:
        return [max(0, min(m - 1, int(math.floor(x + 0.5)))) for x in pos]

    positions: list[list[float]] = []
    velocities: list[list[float]] = []
    for p in range(swarm_size):
        if p == 0:
            pos = [float(x) for x in seed_solution]
        elif p < swarm_size * 0.70:
            sol = seed_solution.copy()
            for j in rng.sample(range(n), rng.randint(1, max(2, n // 4))):
                sol[j] = rng.randrange(m)
            pos = [float(x) + rng.uniform(-0.48, 0.48) for x in sol]
        else:
            pos = [rng.uniform(0, m - 1) for _ in range(n)]
        positions.append(pos)
        velocities.append([rng.uniform(-0.5, 0.5) for _ in range(n)])

    pbest_positions = [p.copy() for p in positions]
    pbest_fitness = [fitness(instance, decode(p)) for p in positions]
    gbest_index = min(range(swarm_size), key=pbest_fitness.__getitem__)
    gbest_position = pbest_positions[gbest_index].copy()
    gbest_fitness = pbest_fitness[gbest_index]
    gbest_assignment = decode(gbest_position)
    history = [gbest_fitness]
    evaluations = swarm_size

    for _ in range(iterations):
        for particle in range(swarm_size):
            for j in range(n):
                velocity = (
                    inertia * velocities[particle][j]
                    + cognitive * rng.random() * (pbest_positions[particle][j] - positions[particle][j])
                    + social * rng.random() * (gbest_position[j] - positions[particle][j])
                )
                velocity = max(-velocity_limit, min(velocity_limit, velocity))
                velocities[particle][j] = velocity
                positions[particle][j] = max(0.0, min(m - 1.0, positions[particle][j] + velocity))
                if rng.random() < exploration_rate:
                    positions[particle][j] = float(rng.randrange(m))
            assignment = decode(positions[particle])
            value = fitness(instance, assignment)
            evaluations += 1
            if value < pbest_fitness[particle] - 1e-9:
                pbest_fitness[particle] = value
                pbest_positions[particle] = positions[particle].copy()
                if value < gbest_fitness - 1e-9:
                    gbest_position = positions[particle].copy()
                    gbest_fitness = value
                    gbest_assignment = assignment.copy()
        history.append(gbest_fitness)
    return SearchResult("PSO", gbest_assignment, gbest_fitness, history, evaluations)


def _roulette(weights: list[float], rng: random.Random) -> int:
    s = sum(weights)
    if s <= 0 or not math.isfinite(s):
        return rng.randrange(len(weights))
    pick = rng.random() * s
    for idx, weight in enumerate(weights):
        pick -= weight
```

```
        if pick <= 0:
            return idx
    return len(weights) - 1


def ant_colony_optimization(instance: Instance, seed: int, iterations: int = 80,
                            ant_count: int = 32, alpha: float = 1.0,
                            beta: float = 3.0, evaporation: float = 0.2,
                            global_elite_weight: float = 1.5) -> SearchResult:
    """ACO with task-machine pheromone, load-aware heuristic and elitist deposit."""
    if ant_count < 1 or iterations < 0 or not (0 < evaporation < 1):
        raise ValueError("Invalid ACO settings")
    rng = random.Random(seed)
    n, m = instance.num_jobs, instance.num_machines
    best_assignment = greedy_assignment(instance)
    best_value = fitness(instance, best_assignment)
    # Warm-start pheromone from same greedy baseline used by GA and PSO.
    pheromone = [[1.3 if best_assignment[j] == k else 1.0 for k in range(m)] for j in range(n)]
    history = [best_value]
    evaluations = 1  # The greedy baseline used to initialize best-so-far.
    durations = instance.durations

    for _ in range(iterations):
        iteration_best: list[int] | None = None
        iteration_value = math.inf
        for ant in range(ant_count):
            # Task difficulty ordering with randomness for solution diversity.
            order = sorted(range(n), key=lambda j: min(durations[j]) * rng.uniform(0.78, 1.22), reverse=True)
            assignment = [0] * n
            loads = [0.0] * m
            for j in order:
                desirability = [
                    pheromone[j][k] ** alpha * (1.0 / (loads[k] + durations[j][k])) ** beta
                    for k in range(m)
                ]
                machine = _roulette(desirability, rng)
                assignment[j] = machine
                loads[machine] += durations[j][machine]
            value = max(loads)
            evaluations += 1
            if value < iteration_value:
                iteration_best, iteration_value = assignment, value
            if value < best_value - 1e-9:
                best_value, best_assignment = value, assignment.copy()

        keep = 1.0 - evaporation
        for row in pheromone:
            for k in range(m):
                row[k] = max(0.04, keep * row[k])
        assert iteration_best is not None
        # Relative scaling keeps deposit magnitudes interpretable across instance sizes.
        for j, machine in enumerate(iteration_best):
            pheromone[j][machine] = min(10.0, pheromone[j][machine] + best_value / iteration_value)
        for j, machine in enumerate(best_assignment):
            pheromone[j][machine] = min(10.0, pheromone[j][machine] + global_elite_weight)
        history.append(best_value)
    return SearchResult("ACO", best_assignment.copy(), best_value, history, evaluations)


ALGORITHMS = {
    "GA": genetic_algorithm,
    "PSO": particle_swarm_optimization,
    "ACO": ant_colony_optimization,
}
```

2. تشغيل التجارب وتصدير النتائج

run_experiments.py

```
"""Run reproducible GA / PSO / ACO comparisons and export tables and figures.

Usage:
    python run_experiments.py
    python run_experiments.py --runs 3 --iterations 40 --population 24
"""
from __future__ import annotations

import argparse
import csv
import json
import math
from pathlib import Path
import platform
import statistics
import time

import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

from scheduler import ALGORITHMS, fitness, generate_instance, greedy_assignment, machine_loads,
theoretical_lower_bound

SIZE_CONFIG = {
    'Small': (20, 3, 501),
    'Medium': (50, 5, 601),
    'Large': (100, 8, 701),
}
ALGO_ORDER = ('GA', 'PSO', 'ACO')
ALGO_COLORS = {'GA': '#2563eb', 'PSO': '#0891b2', 'ACO': '#d97706'}


def csv_write(path, rows, fieldnames):
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open('w', encoding='utf-8-sig', newline='') as handle:
        writer = csv.DictWriter(handle, fieldnames=fieldnames)
        writer.writeheader()
        writer.writerows(rows)


def safe_stdev(values):
    return statistics.stdev(values) if len(values) > 1 else 0.0


def plot_convergence(data_rows, size, baseline, path):
    plt.figure(figsize=(8.4, 4.6))
    for algorithm in ALGO_ORDER:
        points = sorted((x for x in data_rows if x['size'] == size and x['algorithm'] == algorithm),
                        key=lambda r: r['iteration'])
        plt.plot([v['iteration'] for v in points],
                 [v['mean_best_fitness'] for v in points],
                 label=algorithm, color=ALGO_COLORS[algorithm], linewidth=2.0)
    plt.axhline(baseline, color='#64748b', linestyle='--', linewidth=1.2, label='Greedy baseline')
    plt.title(f'Convergence | {size} instance (lower is better)')
    plt.xlabel('Iteration')
    plt.ylabel('Mean best-so-far makespan (minutes)')
    plt.grid(alpha=.20)
    plt.legend()
    plt.tight_layout()
    plt.savefig(path, dpi=180)
    plt.close()


def plot_summary(summary_rows, baselines, metric, ylabel, title, path):
    sizes = list(SIZE_CONFIG)
    width = .23
    base_x = list(range(len(sizes)))
    fig, ax = plt.subplots(figsize=(8.4, 4.65))
    for ai, algorithm in enumerate(ALGO_ORDER):
        subset = [next(r for r in summary_rows if r['size'] == size and r['algorithm'] == algorithm) for size in
sizes]
        x = [v + (ai - 1) * width for v in base_x]
        bars = ax.bar(x, [r[metric] for r in subset], width=width * .92, label=algorithm,
                      color=ALGO_COLORS[algorithm], alpha=.9)
        if metric == 'mean_best_fitness':
            ax.errorbar(x, [r[metric] for r in subset],
                        yerr=[r['std_best_fitness'] for r in subset],
                        fmt='none', ecolor='#111827', capsize=3, linewidth=1)
```

```
    if metric == 'mean_best_fitness':
        for xi, size in enumerate(sizes):
            ax.plot([xi - .39, xi + .39], [baselines[size], baselines[size]],
                    linestyle=':', linewidth=1.4, color='#64748b')
    ax.set_xticks(base_x, sizes)
    ax.set_ylabel(ylabel)
    ax.set_title(title)
    ax.legend()
    ax.grid(axis='y', alpha=.18)
    ax.set_axisbelow(True)
    fig.tight_layout()
    fig.savefig(path, dpi=180)
    plt.close(fig)


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--runs', type=int, default=6, help='Repetitions for each algorithm/size (default: 6)')
    parser.add_argument('--iterations', type=int, default=80, help='Iterations per algorithm (default: 80)')
    parser.add_argument('--population', type=int, default=32, help='Population, swarm size and ant count
(default: 32)')
    parser.add_argument('--output', type=Path, default=Path('results'), help='Output directory')
    args = parser.parse_args()
    if args.runs < 1 or args.iterations < 1 or args.population < 4:
        parser.error('runs>=1, iterations>=1, population>=4 are required')
    out = args.output
    (out / 'plots').mkdir(parents=True, exist_ok=True)
    (out / 'assignments').mkdir(parents=True, exist_ok=True)
    (out / 'instances').mkdir(parents=True, exist_ok=True)
    rows = []
    convergence_rows = []
    summary_rows = []
    best_assignments = {}
    baselines = {}
    lower_bounds = {}
    all_histories = {}

    for size_idx, (size, (jobs, machines, instance_seed)) in enumerate(SIZE_CONFIG.items()):
        instance = generate_instance(size, jobs, machines, instance_seed)
        csv_write(out / 'instances' / f'{size.lower()}_durations.csv',
                  [{'task_id': i+1, **{f'machine_{m+1}_min': duration for m, duration in enumerate(row)}}
                   for i, row in enumerate(instance.durations)],
                  ['task_id'] + [f'machine_{m+1}_min' for m in range(machines)])
        baseline_assignment = greedy_assignment(instance)
        baseline = fitness(instance, baseline_assignment)
        lower_bound = theoretical_lower_bound(instance)
        baselines[size] = baseline
        lower_bounds[size] = lower_bound
        all_histories[size] = {}
        best_assignments[size] = {}
        for algorithm in ALGO_ORDER:
            algorithm_results = []
            histories = []
            best_sol = None
            for run_idx in range(args.runs):
                run_seed = 12000 + size_idx * 1000 + run_idx
                extras = {'GA': {'population_size': args.population},
                          'PSO': {'swarm_size': args.population},
                          'ACO': {'ant_count': args.population}}[algorithm]
                start = time.perf_counter()
                result = ALGORITHMS[algorithm](instance, seed=run_seed, iterations=args.iterations, **extras)
                elapsed = time.perf_counter() - start
                assert abs(result.best_fitness - fitness(instance, result.assignment)) < 1e-6
                assert result.best_fitness <= baseline + 1e-6
                if best_sol is None or result.best_fitness < best_sol.best_fitness:
                    best_sol = result
                record = {
                    'size': size, 'jobs': jobs, 'machines': machines, 'instance_seed': instance_seed,
                    'algorithm': algorithm, 'run': run_idx + 1, 'run_seed': run_seed,
                    'baseline_fitness': baseline, 'lower_bound': round(lower_bound, 4),
                    'best_fitness': round(result.best_fitness, 5),
                    'improvement_pct': round(100 * (baseline - result.best_fitness) / baseline, 4),
                    'runtime_sec': round(elapsed, 6), 'evaluations': result.evaluations,
                    'convergence_iteration': result.convergence_iteration,
                    'machine_loads_std': round(statistics.pstdev(machine_loads(instance, result.assignment)), 4),
                }
                rows.append(record)
                algorithm_results.append(record)
                histories.append(result.history)
            assert best_sol is not None
            best_assignments[size][algorithm] = best_sol.assignment
            all_histories[size][algorithm] = histories
            for iteration in range(args.iterations + 1):
                current = [h[iteration] for h in histories]
                convergence_rows.append({
                    'size': size, 'algorithm': algorithm, 'iteration': iteration,
```

```
                    'mean_best_fitness': round(statistics.mean(current), 5),
                    'std_best_fitness': round(safe_stdev(current), 5),
                })
            values = [r['best_fitness'] for r in algorithm_results]
            times = [r['runtime_sec'] for r in algorithm_results]
            summary_rows.append({
                'size': size, 'jobs': jobs, 'machines': machines, 'algorithm': algorithm,
                'runs': args.runs, 'iterations': args.iterations, 'population': args.population,
                'baseline_fitness': baseline, 'lower_bound': round(lower_bound, 4),
                'mean_best_fitness': round(statistics.mean(values), 4),
                'std_best_fitness': round(safe_stdev(values), 4),
                'min_best_fitness': round(min(values), 4),
                'mean_runtime_sec': round(statistics.mean(times), 5),
                'std_runtime_sec': round(safe_stdev(times), 5),
                'mean_improvement_pct': round(statistics.mean(r['improvement_pct'] for r in algorithm_results),
4),
                'mean_first_best_iteration': round(statistics.mean(r['convergence_iteration'] for r in
algorithm_results), 2),
                'mean_evaluations': round(statistics.mean(r['evaluations'] for r in algorithm_results), 2),
            })
            print(f'{size:6}  {algorithm:3}  best(mean±SD)={statistics.mean(values):8.2f}±
{safe_stdev(values):5.2f}'
                  f'  runtime(mean)={statistics.mean(times):6.3f}s  baseline={baseline:7.2f}', flush=True)
        for algorithm, assignment in best_assignments[size].items():
            loads = machine_loads(instance, assignment)
            csv_write(out / 'assignments' / f'{size.lower()}_{algorithm.lower()}_best.csv',
                      [{'task_id': j+1, 'machine_id': assignment[j]+1,
                        'processing_minutes': instance.durations[j][assignment[j]],
                        'total_load_of_assigned_machine_minutes': round(loads[assignment[j]], 3)}
                       for j in range(jobs)],
                      ['task_id', 'machine_id', 'processing_minutes', 'total_load_of_assigned_machine_minutes'])

    csv_write(out / 'runs.csv', rows, list(rows[0]))
    csv_write(out / 'summary.csv', summary_rows, list(summary_rows[0]))
    csv_write(out / 'convergence.csv', convergence_rows, list(convergence_rows[0]))
    for size in SIZE_CONFIG:
        plot_convergence(convergence_rows, size, baselines[size], out / 'plots' /
f'convergence_{size.lower()}.png')
    plot_summary(summary_rows, baselines, 'mean_best_fitness', 'Mean makespan (minutes)',
                 'Solution quality by problem size (error bars = ±1 SD)', out / 'plots' / 'solution_quality.png')
    plot_summary(summary_rows, baselines, 'mean_runtime_sec', 'Mean runtime (seconds)',
                 'Execution time by problem size', out / 'plots' / 'runtime.png')

    settings = {
        'problem': 'Unrelated parallel machine scheduling; minimize makespan',
        'data_origin': 'Synthetic, seeded; not real company data',
        'matrix_generation': 'round(work_j / machine_speed_m * affinity_jm + setup_jm, 2)',
        'size_config': {k: {'jobs': v[0], 'machines': v[1], 'instance_seed': v[2]} for k,v in
SIZE_CONFIG.items()},
        'runs_per_algorithm_size': args.runs,
        'iterations': args.iterations,
        'population_or_swarm_or_ants': args.population,
        'stochastic_seeds': '12000 + size_index * 1000 + run_index, 0-based indices',
        'algorithms': {
            'GA': {'crossover': 'uniform', 'crossover_rate': 0.9, 'mutation_rate': '1 / jobs per gene',
                   'swap_rate': 0.25, 'elitism': 2, 'tournament_size': 3},
            'PSO': {'inertia': 0.68, 'cognitive': 1.6, 'social': 1.6,
                    'velocity_limit': 2.0, 'exploration_rate': 0.012, 'rounding': 'nearest int and clip'},
            'ACO': {'alpha': 1.0, 'beta': 3.0, 'evaporation': 0.2,
                    'global_elite_weight': 1.5, 'pheromone_floor': 0.04, 'pheromone_ceiling': 10.0},
        },
        'platform': platform.platform(),
        'python_version': platform.python_version(),
        'time_metric': 'wall time; machine/environment dependent',
    }
    (out / 'metadata.json').write_text(json.dumps(settings, ensure_ascii=False, indent=2), encoding='utf-8')
    print(f'\nSaved {len(rows)} runs, {len(summary_rows)} summaries and five plots to {out}')


if __name__ == '__main__':
    main()
```

3. التحقق والاختبارات

tests/test_scheduler.py

```python
"""Run with: python -m unittest discover -s tests -v"""
import itertools
import unittest

from scheduler import (
    Instance, ALGORITHMS, ant_colony_optimization, fitness,
    generate_instance, greedy_assignment, machine_loads, theoretical_lower_bound,
)


class SchedulingTests(unittest.TestCase):
    def setUp(self):
        self.instance = Instance("toy", (
            (5.0, 10.0, 7.0),
            (7.0, 4.0, 8.0),
            (9.0, 8.0, 5.0),
            (4.0, 6.0, 7.0),
            (8.0, 5.0, 9.0),
            (6.0, 8.0, 4.0),
        ))

    def test_objective_and_validation(self):
        a = [0, 1, 2, 0, 1, 2]
        self.assertEqual(machine_loads(self.instance, a), [9.0, 9.0, 9.0])
        self.assertEqual(fitness(self.instance, a), 9.0)
        with self.assertRaises(ValueError):
            fitness(self.instance, [0, 1])
        with self.assertRaises(ValueError):
            fitness(self.instance, [0, 1, 2, 0, 1, 99])

    def test_lower_bound(self):
        optimum = min(fitness(self.instance, a) for a in itertools.product(range(3), repeat=6))
        self.assertLessEqual(theoretical_lower_bound(self.instance), optimum + 1e-9)
        self.assertEqual(optimum, 9.0)

    def test_all_algorithms_feasible_reproducible_and_monotone(self):
        greedy = fitness(self.instance, greedy_assignment(self.instance))
        for name, algorithm in ALGORITHMS.items():
            with self.subTest(algorithm=name):
                res = algorithm(self.instance, seed=42, iterations=12, **{
                    "GA": {"population_size": 12},
                    "PSO": {"swarm_size": 12},
                    "ACO": {"ant_count": 12},
                }[name])
                second = algorithm(self.instance, seed=42, iterations=12, **{
                    "GA": {"population_size": 12},
                    "PSO": {"swarm_size": 12},
                    "ACO": {"ant_count": 12},
                }[name])
                self.assertEqual(res.history, second.history)
                self.assertEqual(res.assignment, second.assignment)
                self.assertEqual(len(res.history), 13)
                self.assertEqual(len(res.assignment), self.instance.num_jobs)
                self.assertAlmostEqual(res.best_fitness, fitness(self.instance, res.assignment))
                self.assertLessEqual(res.best_fitness, greedy + 1e-8)
                self.assertTrue(all(x >= y - 1e-8 for x, y in zip(res.history, res.history[1:])))

    def test_generation_is_reproducible(self):
        a = generate_instance("same", 20, 4, 123)
        b = generate_instance("same", 20, 4, 123)
        self.assertEqual(a, b)
        self.assertEqual(len(a.durations), 20)
        self.assertEqual(len(a.durations[0]), 4)

    def test_zero_iterations(self):
        result = ant_colony_optimization(self.instance, 5, iterations=0)
        self.assertEqual(len(result.history), 1)
        self.assertAlmostEqual(result.best_fitness, fitness(self.instance, result.assignment))


if __name__ == "__main__":
    unittest.main()
```

نهاية الشيفرة المصدرية. نتائج التجارب والتقرير الأكاديمي وتعليمات README متاحة داخل ملف المشروع المضغوط.